LLC: Listes Linéaires Chaînées

- Définition d'une LLC
- Implémentation d'une LLC
- Opérations sur les LLCs

Rado Rakotonarivo (Cours d'Asma Gabis)
rado.rakotonarivo@efrei.fr

P1 – 2024/2025

01

Les LLC

01

Les LLCs

Exemple introductif

Problème :

Trouver tous les nombres premiers de **1** à **n** et les stocker en mémoire.

Solution 1 : Utiliser un tableau

- **Si tableau statique :** Quelle taille physique utiliser étant donné que **n** n'est pas connu ?
- **Si tableau dynamique** : même si **n** est connu, quelle taille du tableau réserver ?

Existe-il un nouveau moyen de stockage ?

Les LLCs

Objectifs recherchés

- Allouer de l'espace mémoire en fonction du besoin
- Simuler des problèmes réels
  - Stocker les nombres premiers au fur et à mesure
  - Stocker les malades qui arrivent aux urgences d'un hôpital
  - Gérer une file d'attente dans un guichet
  - ….

Une nouvelle structure de données

Une Liste Linéaire Chaînée (LLC)

Définition

Une LLC est une structure de données composée de plusieurs éléments appelés **cellules** chaînés entre eux.

Une cellule dans une LLC ⇔ Case dans un tableau

Les LLCs

Fonctionnement

Le stockage dans les tableaux nécessite que toutes les cases soient dans une zone contigüe se suivant les unes les autres.

	T[0]	T[1]	T[2]	…	…	…
T	val1	val2	val3	…	…	…

Dans le cas d'une LLC, ceci n'est plus nécessaire. Les **cellules** stockant les valeurs sont réparties dans différentes zones mémoire.

				val2		
Mémoire	val3				val6	
			val1		val5	
		val4				

Comment faire pour relier les cellules entre eux ?

Les LLCs

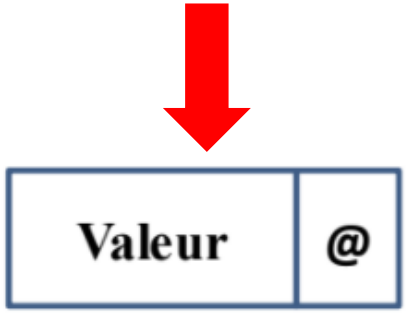
Chaînage

Pour relier les cellules entre eux , l'idée est que **chaque cellule** doit également stocker en mémoire l'information sur la position mémoire de la prochaine cellule de la liste, appelée **successeur**.

➔ Chaque cellule stocke alors deux informations:

- ➔ La valeur
- ➔ L'adresse mémoire de son successeur

Deux informations de deux types différents
➔ **Nécessité d'utiliser une structure**



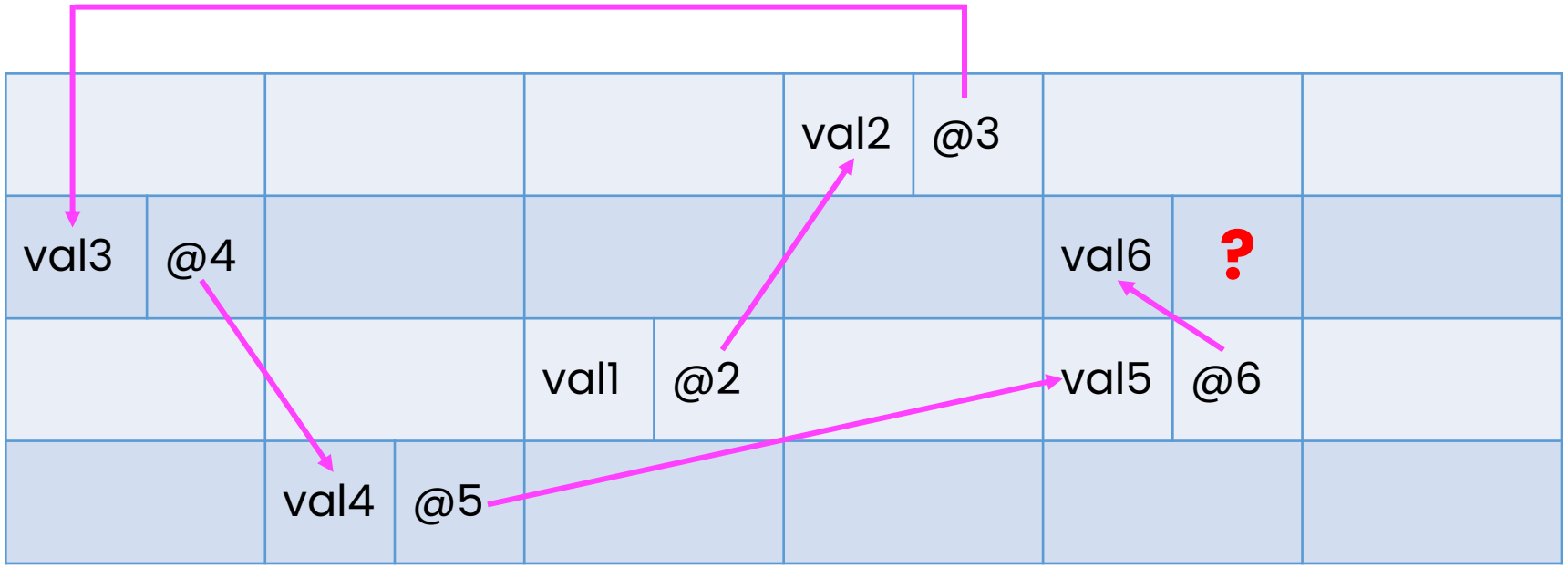
une cellule d'une LLC est toujours une structure avec deux champs :

un champ **Valeur** : contenant l'information utile

un champ **Adresse** : donnant l'adresse de la prochaine cellule

Les LLCs

Chaînage



D'où le nom : **chaînage linéaire**

Comment repérer la première cellule ?

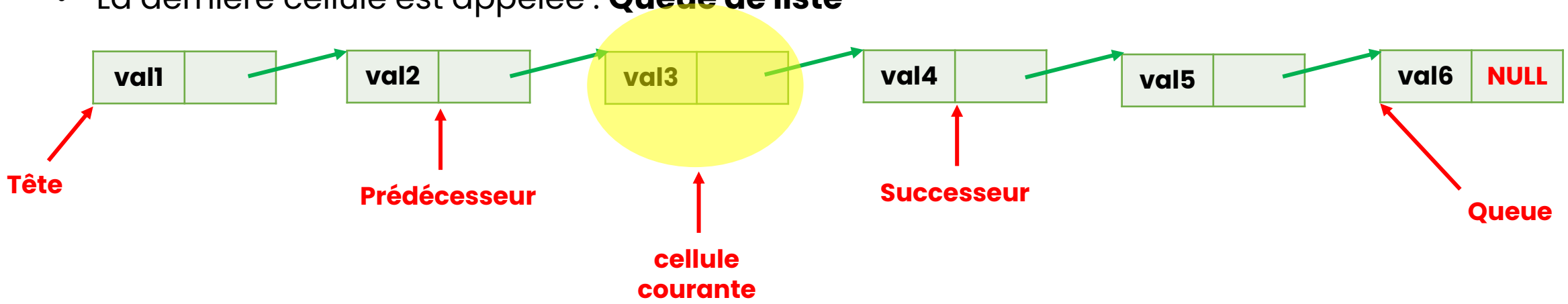
Quelle adresse pour le successeur de la dernière cellule ?

Les LLCs

Caractéristiques

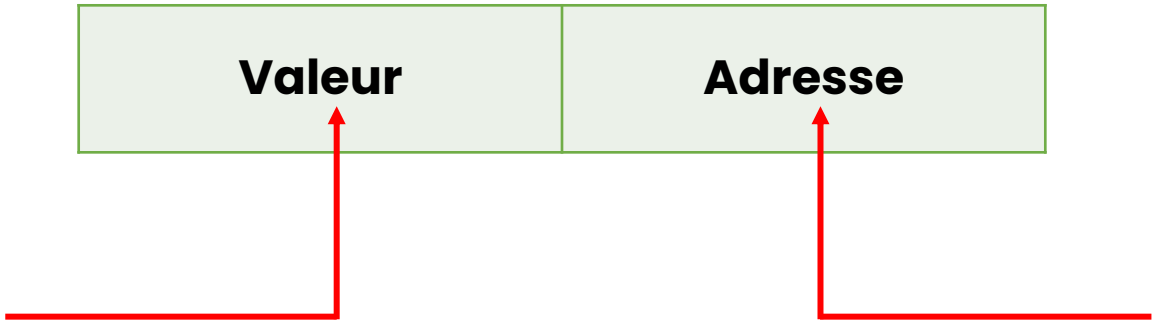
Une LLC est caractérisée par :

- **l'adresse de la première cellule :** Elle doit toujours être sauvegardée dans une variable pour pouvoir manipuler la liste.
- **Si une cellule a l'adresse NULL comme adresse de son successeur :** cette cellule est nécessairement la dernière cellule de la liste.
- La première cellule est appelée : **Tête de liste**
- La dernière cellule est appelée : **Queue de liste**



Les LLCs

Contenu



Il est possible de stocker dans le champ Valeur des données :
- De types primitifs
- De types structurés

Champs contenant l'adresse de la cellule suivante : une cellule de même type que le courant

02

Implémentation d'une LLC

02

Implémentation d'une LLC

Définition de la structure d'une cellule

```
struct s_cell
{
        type1 champ_valeur ;
        type2 champ_adresse ;
} ;
```

type1 : type de l'information utile stockée dans la LLC

type2 : adresse vers une cellule de même type

```
struct s_cell
{
        type1 value ;
        struct s_cell *next;
} ;
```

Le pointeur est de retour

Auto référence

Implémentation d'une LLC

Définition de la structure d'une cellule

Pour faire simple, supposons que la liste contient des valeurs de type **int**

```
struct s_cell
{
    int value;
    struct s_cell *next;
};
/*-Définition du type cellule*/
typedef struct s_cell t_cell;


/*-Définition du type liste-*/
typedef t_cell* t_list;


/*-Déclaration d'une liste vide-*/
t_list L=NULL;//ou t_cell* l=NULL;
```

Implémentation d'une LLC

Opérations sur les LLCs

Les opérations principales effectuées sur les LLCs sont :

- **Le parcours** ➔ Déterminer la condition d'arrêt + ne pas perdre le pointeur de tête de liste
- **L'insertion d'une cellule** ➔ Repérer l'endroit et effectuer l'ajout
- **La suppression d'une cellule** ➔ Repérer la cellule à supprimer et le retirer de la LLC

03

Opérations sur les LLCs

03

Opérations sur les LLCs

Besoin

Pour effectuer n'importe quelle opération sur une LLC, nous avons besoin de :

- **Allouer les cellules** ➔ Utiliser le **malloc** en langage C
- **Libérer les cellules** ➔ Utiliser le **free** en langage C
- **Vérifier si la LLC n'est pas vide** ➔ vérifier si la première cellule n'est pas NULL

Opérations sur les LLCs

Création d'une cellule

```
t_cell *m ;
m = (t_cell *) malloc (sizeof(t_cell ));
m->value = 3 ;
m->next = NULL;
```

Création d'une liste à une cellule

```
/*-Déclaration d'une liste vide-*/
t_list L=NULL;//ou cellule* L=NULL;

t_cell *m;
m = (t_cell *) malloc (sizeof(t_cell));
m->value = 3 ;
m->next = NULL;

L = m;
```

Libérer une cellule

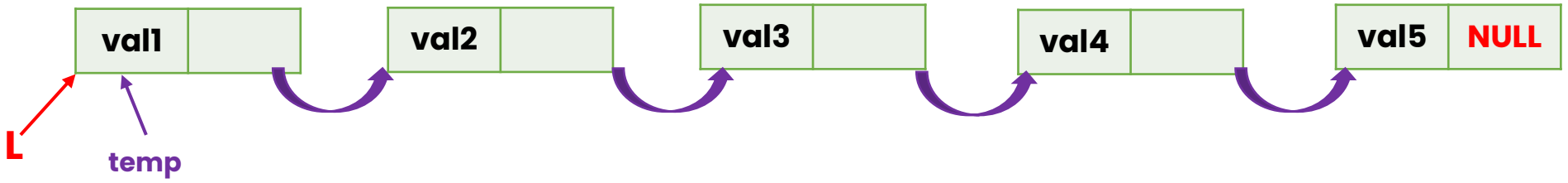
```
free(m);
m = NULL;
```

Opérations sur les LLCs

Parcourir une LLC

Pour parcourir une LLC, nous avons besoin de :

1. Vérifier que la LLC **L** existe ➔ non vide
2. Initialiser un pointeur **tmp** (qui sert de curseur) à la tête de liste
3. Parcourir les cellules une à une de la **tête** vers la **queue** tant que **tmp ≠ NULL**
4. Effectuer l'opération souhaitée: affichage (par exemple)



Opérations sur les LLCs

Parcourir une LLC

```
// on utilise un pointeur temporaire tmp qu'on pointe
// sur la première cellule de la liste qui est pointé par l
t_cell * tmp ;
if ( L != NULL)
{
    tmp = L;
    while(tmp != NULL)
    {
        // tant que la cellule suivante existe, on pointe tmp vers celui-ci
        tmp = tmp->next;
    }
}
```

Opérations sur les LLCs

Ajouter une cellule à une LLC

3 cas de figure :

1. Ajouter une cellule en tête de liste

2. Ajouter une cellule à la fin de la liste

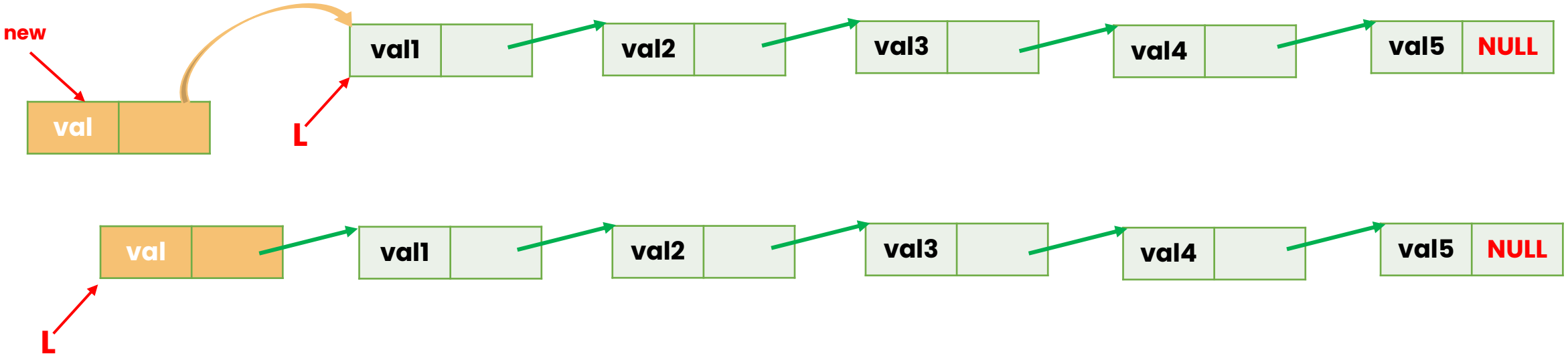
3. Ajouter une cellule au milieu de la liste

Opérations sur les LLCs

Ajouter une cellule en tête de liste

Étapes : Pour une liste L

1. Création une nouvelle cellule **new** avec la valeur souhaitée
2. Associer à l'adresse du successeur de **new** l'adresse de **L** (la tête courante)
3. Déplacer **L** vers **new** pour mettre à jour la nouvelle tête de liste



Opérations sur les LLCs

Ajouter une cellule en tête de liste

```
/*On crée une nouvelle cellule*/
t_cell *new = (t_cell*)malloc(sizeof(t_cell));

/* On assigne la valeur val au champ valeur de la nouvelle cellule */
new->value = 4;

/*On assigne l'ancienne tête de liste comme étant la suivante de la nouvelle cellule*/
new->next = L;

/*On met à jour la tête de liste (c'est désormais new)*/
L = new ;
```

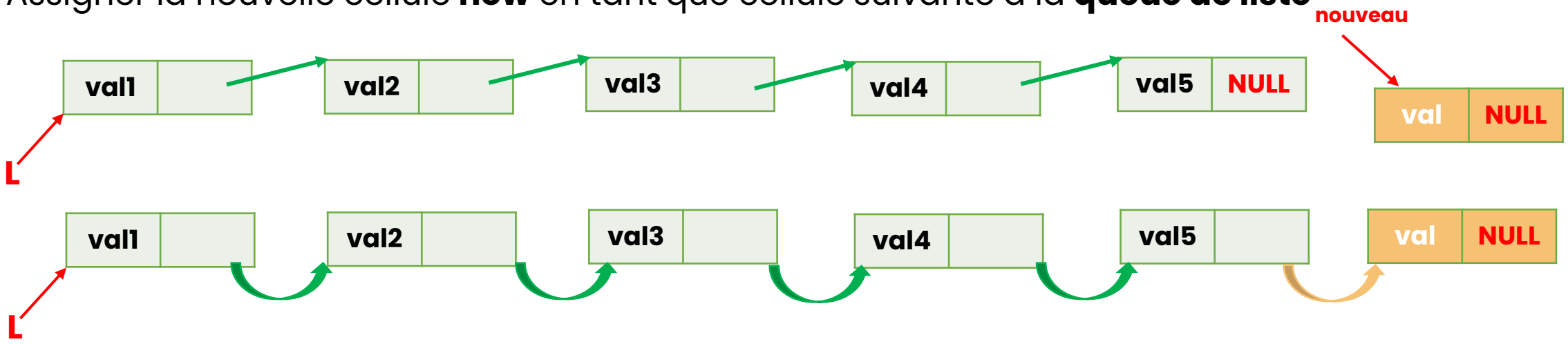
Notez que ce bout de code fonctionne même si la liste L est initialement vide

Opérations sur les LLCs

Ajouter une cellule en fin de liste

Étapes : Pour une liste **L** non vide

1. Création d'une nouvelle cellule **new** avec la valeur souhaitée
2. Parcourir la liste **L** jusqu'à atteindre la **queue de liste**
3. Assigner la nouvelle cellule **new** en tant que cellule suivante à la **queue de liste**



Opérations sur les LLCs

Ajouter une cellule en fin de liste

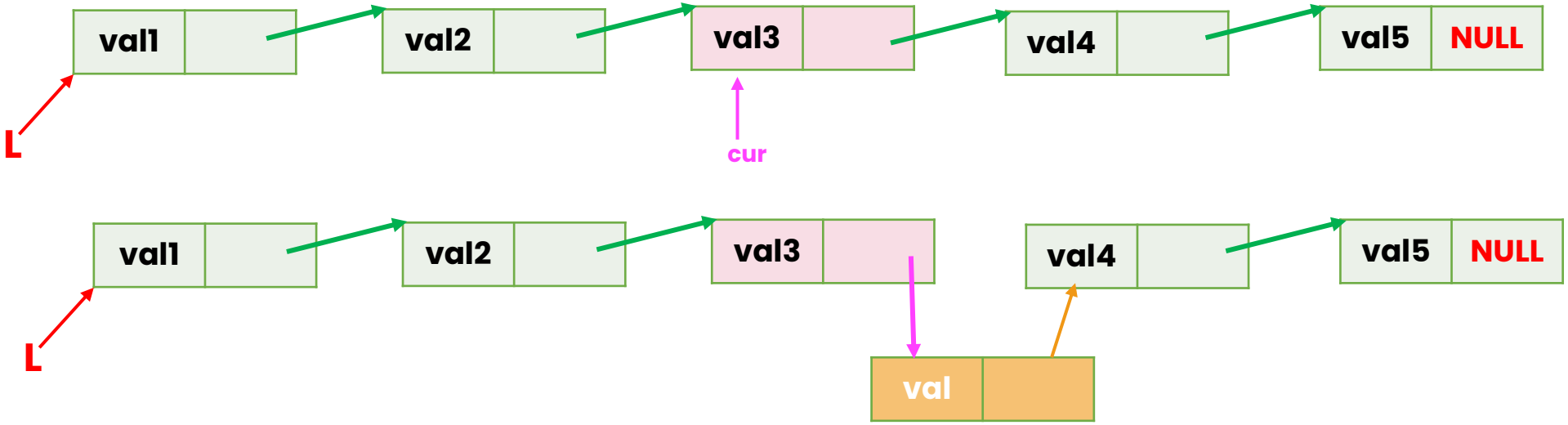
```
/*On crée une nouvelle cellule */
t_cell *new = (t_cell *)malloc(sizeof(t_cell));
new->value = 4;
new->next = NULL;
if(L == NULL){
    /*Si la liste est vide il suffit de la positionner sur la cellule créé*/
    L = new;
}
else {
    /*Sinon, on parcourt la liste à l'aide d'un curseur tant qu'on n'a pas atteint la dernière cellule*/
    t_cell * tmp=L;
    while(tmp->next != NULL) /*tmp->next == NULL signifie que tmp pointe sur la dernière cellule*/
    {
        tmp = tmp->next;
    }
    /*On relie ensuite l'acienne queue de liste à new (qui sera désormais la queue de liste)*/
    tmp->next = new;
}
```

Opérations sur les LLCs

Ajouter une cellule au milieu de la liste

Étapes : Pour une liste **L** non vide

1. Création d'une nouvelle cellule **new** avec la valeur souhaitée
2. Parcourir la liste **L** jusqu'à repérer l'endroit de l'insertion (indiquée par un curseur **cur**)
3. Mettre à jour le successeur de **new** comme étant le successeur de **cur**
4. Mettre à jour le successeur de **cur** comme étant **new**



Opérations sur les LLCs

Supprimer une cellule d'une LLC

3 cas de figure :

1. Supprimer la tête de liste
2. Supprimer la queue de la liste
3. Supprimer une cellule au milieu de la liste

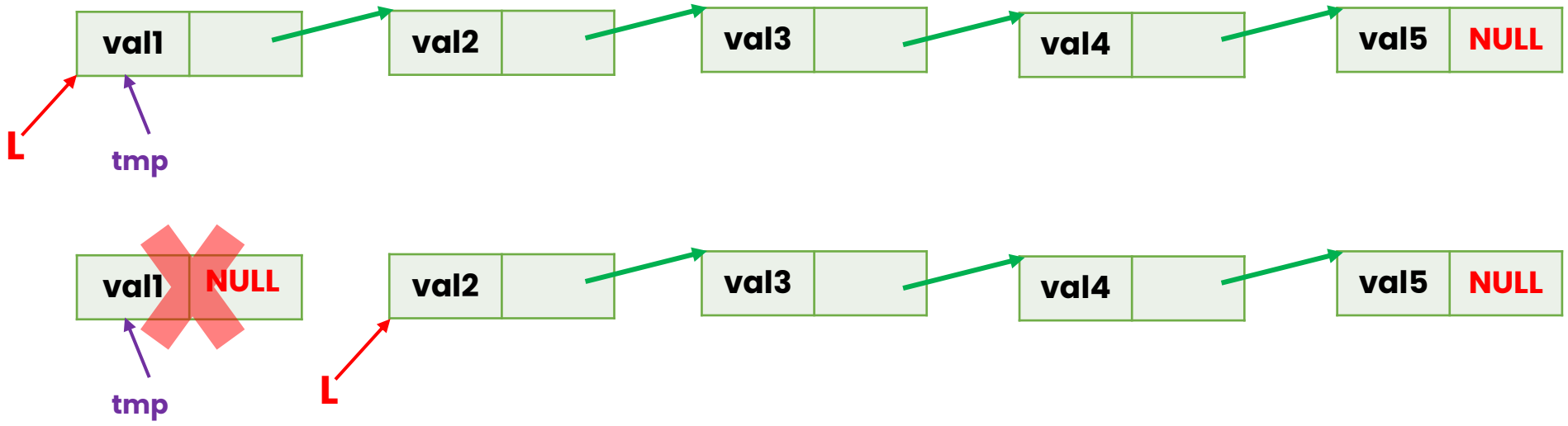
Opérations sur les LLCs

Supprimer la tête d'une LLC

Étapes : pour une liste L non vide

1. Créer un pointeur **tmp** et le positionner sur la tête **L**
2. Déplacer la tête de liste sur son successeur
3. Mettre le successeur du cellule **tmp** à **NULL** puis libérer **tmp**

```
if (L != NULL)
{
    t_cell* tmp = L;
    L = L->next;
    tmp->next = NULL;
    free(tmp);
}
```



Opérations sur les LLCs

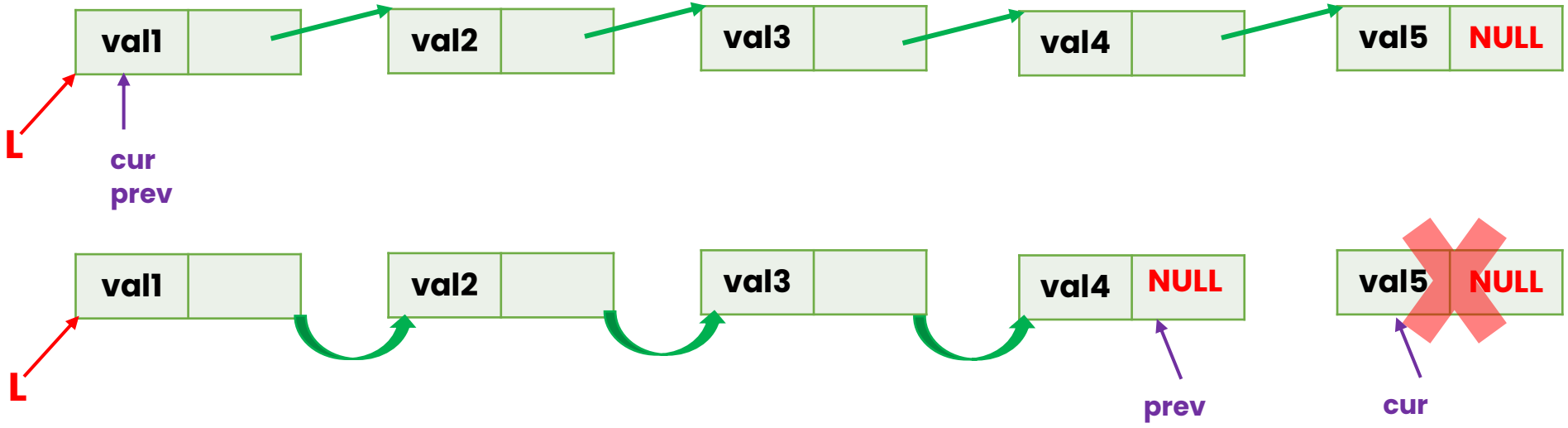
Supprimer la queue d'une LLC

Étapes : pour une liste **L** non vide

1. Utiliser deux pointeurs:
   - un pointeur **cur** qui pointe sur la cellule courante
   - un autre pointeur **prev** qui pointe sur la cellule précédente (de **cur**)
2. Tant que **cur** ne pointe pas sur la dernière cellule, on déplace ces deux pointeurs d'une position
3. A la sortie de la boucle :
   - **cur** pointe pas sur la dernière cellule
   - **prev** pointe sur l'avant-dernier cellule
4. Mettre à jour le successeur de **prev** à **NULL** puis libérer **cur**

Opérations sur les LLCs

Supprimer la queue d'une LLC



Opérations sur les LLCs

Supprimer la queue d'une LLC

```
/* Si la liste n'est pas vide */
if(L != NULL){
    /*Si la liste contient une seule cellule*/
    if(L->next == NULL){
        /*On le libère la cellule (la liste est maintenant vide)*/
        free(L);
        L = NULL;
    }
    else {
        /*Si la liste contient au moins deux cellules*/
        t_cell * cur = L;
        t_cell * prev = L;
        /* Tant qu'on n'est pas à la dernière cellule */
        while(cur->next != NULL)
        {
            /* prev prend la place de cur*/
            prev = cur;
            /* on déplace cur sur la cellule suivante*/
            cur = cur->next;
        }
        /* A la sortie de la boucle, cur pointe sur la dernière cellule, et prev sur l'avant-dernière.*/
        /* prev est désormais la queue de liste, elle prend comme next l'adresse NULL*/
        prev>next = NULL;
        free(cur);
    }
}
```

Opérations sur les LLCs

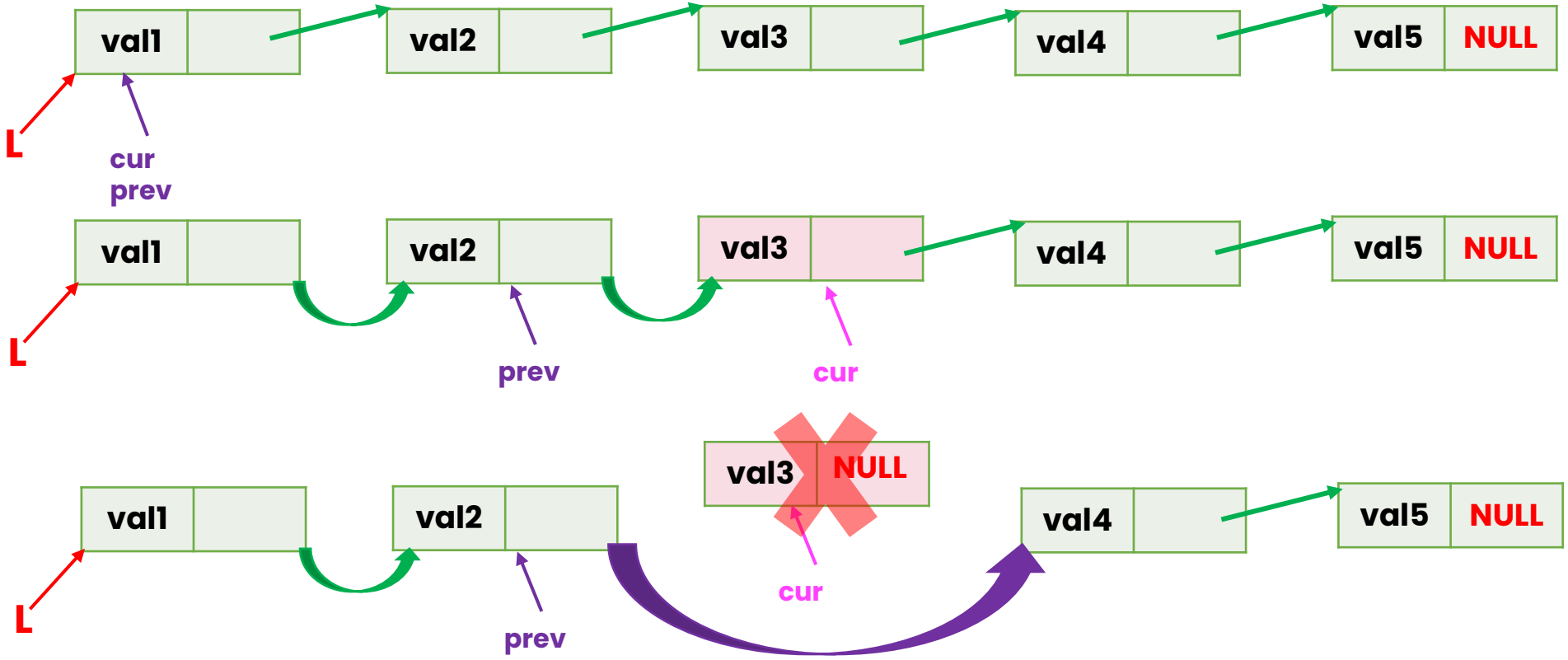
Supprimer une cellule au milieu de la LLC

Étapes : pour une liste **L** non vide

1. Utiliser deux pointeurs:
   - un pointeur **cur** qui pointe sur la cellule courante
   - un autre pointeur **prev** qui pointe sur la cellule précédente (de **cur**)
2. Tant qu'on **cur** ne pointe pas sur la cellule à supprimer, on déplace ces deux pointeurs d'une position
3. A la sortie de la boucle :
   - **cur** pointe sur la cellule à supprimer
   - **prev** pointe sur la cellule qui précède celui à supprimer
4. Mettre à jour le successeur de **prev** comme étant le successeur de **cur**
5. Mettre à jour le successeur de **cur** à **NULL** puis la libérer

Opérations sur les LLCs

Supprimer une cellule au milieu de la LLC



04

Conclusions

04

Conclusions

LLC Vs Tableaux

Les listes sont des structures de données qui permettent, au même titre que les tableaux, de garder en mémoire des données de même type en respectant un certain ordre.

Les listes chaînées, par rapport aux tableaux, possèdent **les avantages** suivants:

- 👍 Elles sont dynamiques et n'occupent qu'un minimum d'espace lorsqu'elles sont vides
- 👍 L'ajout ou le retrait d'une donnée dans une liste ordonnée ne nécessite pas le déplacement des autres données de la liste
- 👍 L'ajout d'une donnée ne demande qu'une réservation minimale de mémoire (la taille d'une donnée et celle d'un ou deux pointeur(s)), alors que pour un tableau dynamique il faut pouvoir allouer un bloc de mémoire.

Conclusions

LLC Vs Tableaux

Cependant, les listes chainées amènent aussi les **désavantages** suivants:

👎 Le code nécessaire pour les gérer est plus complexe que celui pour gérer des tableaux dynamiques

👎 Elles ne permettent pas d'accéder directement à un élément, ce qui empêche entre autres la recherche dichotomique.

Conclusions

Réussir mes programmes sur les LLCs

Pour réussir son programme sur les LLCs :

- Schématiser (dessiner) la LLC
- Dessiner les pointeurs sur les cellules
- Observer leur déplacement